Generative AI of Auto Insurance Pricing Test Data

Author: Douglas Fulcher

Bellevue University

Course Number: DSC670 T301 Spring 2025 – Advanced Use Cases in Generative AI

Frank Neugebauer

March 23, 2025

Milestone 1: Generative AI in Auto Insurance Pricing Test Data

Description of the Idea

Auto insurance pricing algorithms rely heavily on robust, realistic, and varied test data to accurately assess risk, set appropriate premiums, and predict claims. Creating comprehensive auto insurance test datasets is often manual, time-consuming, and prone to biases or insufficient data variety. The proposed project seeks to leverage Generative Artificial Intelligence (GAI) to develop a specialized model capable of generating realistic, diverse, and statistically representative synthetic test data tailored explicitly for auto insurance pricing algorithms.

Main Goals

The main objectives of this project include:

- Automating the generation of comprehensive and diverse synthetic auto insurance test datasets.
- Ensuring generated test data accurately mirrors realistic distributions and correlations in real-world auto insurance data.
- Enhancing the efficiency and effectiveness of testing auto insurance pricing models.
- Minimizing biases or coverage gaps in manually generated datasets.
- Accelerating experimentation and iterative improvements in auto insurance model development.

Initial Approach

Research and Data Collection

The first step involves identifying critical attributes of auto insurance pricing, including vehicle type, driving history, demographics, claims frequency, and location-specific risks. Subsequently, publicly available and industry-specific auto insurance datasets suitable for initial training will be collected from sources such as Kaggle.

Exploration of Generative AI Models

Next, various generative AI techniques, such as GPT-based models and Variational Autoencoders, will be investigated to determine their effectiveness in handling structured,

tabular data. An AI model demonstrating superior capability in generating synthetic structured data relevant to auto insurance will then be selected and evaluated.

Fine-tuning and Validation

The selected generative AI model will be fine-tuned using auto insurance-specific data attributes. The data will be validated by analyzing the outputs of the Generative AI model to determine if it creates a synthetic dataset that could be used to price an auto insurance policy. While the synthetic datasets created by the GAI model should be sufficient to price an auto insurance policy, they will not be run through a pricing algorithm as part of this project.

The outlined approach will adapt as further insights into generative AI technologies and specific auto insurance data requirements are acquired.

Milestone 2: Generative AI Selection and Prompt Experiments

Updated Problem Statement

Automated auto insurance pricing models require accurate, varied, and statistically representative test datasets to optimize performance and mitigate biases. Manual generation of these datasets often lacks variety, realism, and comprehensive statistical coverage. Therefore, leveraging Generative AI (GAI) for automating the production of synthetic test datasets offers significant potential for improving model development efficiency and reliability.

Chosen Generative AI Types

Based on exploration of generative techniques, GPT-based text-to-text models, specifically GPT-4o, would be a good fit to address this opportunity. The proven capability of this model in structured data extraction and synthetic data generation tasks, provides a balance between effectiveness and affordability.

Prompt Experiments

Prompt 1 – Zero Shot

Generate a realistic dataset in nested json format containing:

- policy level details such as policy number, named insured marital status, named insured age, and named insured insurance tenure
- driver level details such as driver id, driver age, driving experience (in months), driver marital status, driver annual miles driven
- incident level details such as incident id, incident date, incident type (conviction, violation, or accident), incident fault (at-fault, not at fault), incident status (open,

closed), claim incident paid amount, claim incident type (injury to others, property damage to others, insured property damage)
- vehicle level details such as vehicle id, vehicle cost new, vehicle body style, vehicle safety features (airbag types), vehicle risk features (torque, customizations, etc)
- vehicle coverages (coverages purchased for the vehicle, e.g., Bodily Injury, Comprehensive, Uninsured Motorist, etc), coverage limits selected (per person and per accident), deductibles, etc.
- Create keys to join the data across levels.

Prompt 2 – Zero Shot

Generate a synthetic insurance pricing dataset in structured JSON format. The dataset should include comprehensive data at policy, driver, incident, vehicle, and vehicle coverage levels with realistic and varied entries. Each level should maintain hierarchical consistency using foreign keys to link data appropriately.

Include the following specific details in your dataset:

Policy Level Features:

- policy_id: unique identifier for each policy
- insured_id: unique identifier for each insured
- insured_age: integer (age in years at policy start)
- insured_marital_status: "single", "married", "divorced", "widowed"
- insured_location_code: valid CBG or ZIP code
- insured_driving_experience: integer (number of months since first licensed)
- insured_insurance_tenure: integer (number of months the insured has carried insurance)

Driver Level Features (linked to policies via policy_id):

- driver_id: unique identifier for each driver
- driver_age: integer (age in years at policy start)
- driver_experience: integer (number of months since first licensed)
- driver_marital_status: "single", "married", "divorced", "widowed"
- driver_annual_mileage: integer (annual miles driven)

Incident Level Features (linked to drivers via driver_id):

- incident_id: unique identifier for each incident
- incident_date: date (YYYY-MM-DD)
- incident_fault_indicator: "at-fault", "not-at-fault"
- incident_claim_paid_amt: decimal (amount paid on claim)
- incident_claim_type: "comp only", "liability only", "liability plus injury", "other"

Vehicle Level Features (linked to policies via policy_id):

- vehicle_id: unique identifier (valid VIN format)

- vehicle_cost_new: decimal (cost when purchased)
- vehicle_body_style: "2 door sedan", "4 door sedan", "pickup", "van", etc.
- vehicle_airbags: "driver side only", "passenger side only", "driver and passenger", "side impact"

Vehicle Coverages (linked to vehicles via vehicle_id):

- vehicle_coverage: "BI", "PD", "COMP", "COLL", "RR", "TL", "MP", "EB", "UMBI", "UMPD", "UIMBI", "UIMPD"
- vehicle_coverage_limit_per_person: "50", "100", "200", "500", "1M", "NA"
- vehicle_coverage_limit_per_accident: "100", "300", "500", "1M", "NA"
- vehicle_coverage_deductible: integer (100, 250, 500, 1000)

Ensure realistic distribution and variability across all features.

Prompt 2 – One Shot

Below is an example of structured JSON representing synthetic insurance pricing data. Create another similar synthetic dataset containing policy, driver, incident, vehicle, and vehicle coverage information using the same JSON structure.

Example JSON:


```
{
  "policies": [
    {
      "policy_id": "POL123456",
      "insured_id": "INS123456",
      "insured_age": 45,
      "insured_marital_status": "married",
      "insured_location_code": "10001",
      "insured_driving_experience": 300,
      "insured_insurance_tenure": 120,
      "drivers": [
        {
          "driver_id": "DRV123456",
          "driver_age": 45,
          "driver_experience": 300,
          "driver_marital_status": "married",
          "driver_annual_mileage": 15000,
          "incidents": [
            {
              "incident_id": "INC123456",
              "incident_date": "2024-05-15",
              "incident_fault_indicator": "at-fault",
              "incident_claim_paid_amt": 5000.00,
              "incident_claim_type": "liability plus injury"
            }
          ]
        }
      ],
      "vehicles": [
        {
          "vehicle_id": "1HGBH41JXMN109186",
```

```
            "vehicle_cost_new": 30000.00,
            "vehicle_body_style": "4 door sedan",
            "vehicle_airbags": "driver and passenger",
            "vehicle_coverages": [
              {
                "vehicle_coverage": "BI",
                "vehicle_coverage_limit_per_person": "100",
                "vehicle_coverage_limit_per_accident": "300",
                "vehicle_coverage_deductible": 500
              }
            ]
          }
        ]
      }
    ]
}
```


Generate another dataset similar to this structure, ensuring the uniqueness of IDs and realistic variation of values.

Prompt 4 – Chain of Thought

Let's systematically build a synthetic insurance pricing dataset step-by-step, ensuring logical consistency and realistic variability. Follow each step carefully to create a structured JSON dataset.

Step 1: Policy-Level Information

- First, define a unique `policy_id` for each insurance policy.
- Assign each policy an `insured_id` that uniquely identifies the insured individual.
- Determine `insured_age` in integer years (typical range from 16 to 80 years old).
- Specify the `insured_marital_status` from "single", "married", "divorced", or "widowed".
- Use realistic `insured_location_code` values (CBG or ZIP code).
- Assign `insured_driving_experience` in integer months (e.g., from 12 months to 720 months).
- Assign `insured_insurance_tenure` in integer months (e.g., from 1 month to 360 months).

Step 2: Driver-Level Information (Linked to policies via `policy_id`)

- Create a `driver_id` unique to each driver.
- Define `driver_age` (typical range from 16 to 80 years).
- Assign `driver_experience` in integer months (must logically align with the driver's age).
- Select a `driver_marital_status` consistent with the statuses listed above.
- Assign realistic `driver_annual_mileage` (typically ranging between 3,000 and 25,000 miles).

Step 3: Incident-Level Information (Linked to drivers via `driver_id`)

- Assign each incident a unique `incident_id`.

- Provide an `incident_date` using YYYY-MM-DD format.
- Indicate `incident_fault_indicator` clearly as "at-fault" or "not-at-fault".
- Set a realistic `incident_claim_paid_amt` as a decimal value.
- Specify `incident_claim_type`: "comp only", "liability only", "liability plus injury", or "other".

Step 4: Vehicle-Level Information (Linked to policies via `policy_id`)

- Generate a unique `vehicle_id` using standard VIN formatting.
- Assign a realistic `vehicle_cost_new` in decimal format (range typically from $15,000 to $100,000).
- Select a `vehicle_body_style` from realistic categories (e.g., "2 door sedan", "4 door sedan", "pickup", "van").
- Clearly state the `vehicle_airbags` feature ("driver side only", "passenger side only", "driver and passenger", "side impact").

Step 5: Vehicle Coverage Information (Linked to vehicles via `vehicle_id`)

- Specify `vehicle_coverage` from options: "BI", "PD", "COMP", "COLL", "RR", "TL", "MP", "EB", "UMBI", "UMPD", "UIMBI", "UIMPD".
- Provide realistic values for `vehicle_coverage_limit_per_person` ("50", "100", "200", "500", "1M", "NA").
- Assign realistic values to `vehicle_coverage_limit_per_accident` ("100", "300", "500", "1M", "NA").
- Set realistic `vehicle_coverage_deductible` amounts (100, 250, 500, or 1000).

Ensure each step logically aligns with previous steps, maintaining consistency and realistic scenarios throughout the dataset.

Prompt 5 – Fine-Grained Demographic Data Generation

Generate a comprehensive synthetic dataset in structured JSON format designed for demographic analysis in relation to driving history. The dataset must reflect realistic demographic distributions and their correlations to driving behaviors, accidents, and incidents. Include the following detailed demographic attributes:

1. Demographic Details:

- individual_id (unique identifier)
- age (integer, realistic range 16-85)
- gender (male, female, non-binary)
- marital_status (single, married, divorced, widowed)
- education_level (high school, bachelor's, master's, doctoral, none)
- annual_income (integer, realistic income range from $15,000 to $250,000)

2. Location Details:

- individual_id (foreign key to demographics)
- location_type (urban, suburban, rural)
- state
- county
- ZIP_code
- Census_Block_Group (CBG)

3. Driving History Details:

- individual_id (foreign key to demographics)
- years_licensed (integer, 1 to 70)
- annual_mileage (integer, range from 1,000 to 30,000 miles)
- primary_vehicle_type (sedan, SUV, truck, motorcycle, electric vehicle, hybrid)

4. Incident and Accident Details (reflecting correlation with demographic and driving history):

- incident_id (unique identifier)
- individual_id (foreign key)
- incident_type (accident, moving violation, non-moving violation)
- incident_severity (minor, moderate, severe)
- incident_date (date, YYYY-MM-DD)
- incident_fault (at-fault, not-at-fault)
- insurance_claim_amount (integer, realistic claim amounts from $0 to $50,000)

Ensure the data demonstrates realistic correlations, such as:

- Younger drivers (<25 years old) typically have higher incident rates.
- Higher income individuals may drive more expensive vehicles with potentially different incident profiles.
- Urban areas may have more frequent but lower-severity incidents compared to rural areas.
- Education level and marital status influencing driving patterns and incident occurrences.

Generate a dataset suitable for robust demographic and predictive analysis applications.

General Fine-Tuning Strategy

To fine-tune the GPT-4o model for generating synthetic auto insurance data, the following specific strategy and methods will be employed:

1. Data Collection and Labeling:
    a. Collect auto insurance datasets from public sources (e.g., Kaggle) and industry repositories.
    b. Manually label and validate subsets of the data, clearly defining attributes (customer demographics, vehicle information, claims history, etc.).
2. Dataset Preparation:

    a. Preprocess the data, ensuring consistency in structure and format suitable for model ingestion.
    b. Segment the data into training, validation, and testing sets to evaluate model accuracy and generalizability.
3. API-based Fine-Tuning:
    a. Utilize OpenAI's fine-tuning APIs to feed labeled datasets into GPT-4o.
    b. Configure hyperparameters such as learning rates, epochs, and batch sizes through systematic experimentation and validation.
4. Iterative Model Evaluation:
    a. Evaluate the fine-tuned model outputs against real-world benchmarks and distributions.
    b. Perform quantitative assessments of model accuracy, realism, and structural integrity of synthetic data.
5. Refinement and Validation:
    a. Iterate the fine-tuning process based on evaluation feedback, adjusting labeled data and fine-tuning parameters accordingly.
    b. Engage human expert review periodically to ensure domain-specific accuracy and reliability.
6. Ethical and Regulatory Compliance:
    a. Incorporate ethical considerations into model fine-tuning, explicitly addressing privacy concerns, data bias, and transparency.
    b. Align fine-tuning processes and outcomes with regulatory requirements (e.g., GDPR, CCPA).

Milestone 2 Concluding Thoughts

I'm still not 100% sure that I understand exactly how this fine-tuning will occur. While I did look ahead to week 7, I am not clear on how labeled data can be passed to the fine-tuning APIs to improve GPT-4o's performance on the desired task. I'm hoping that I can locate insurance premium related datasets online that can provide sufficient samples to GPT on what factors are most likely to influence insurance premiums. I believe that better examples will improve the above prompts and allow me to get closer to real-world examples that the GPT models can work with to generate better results. Finally, I'm also concerned about cost. So, while I would like to use at least GPT-4o, I may wind up using o3-mini which has a lower overall cost.